

Lab 0 — Toolchain: Hello, ESP32-S3

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	1-2 hrs
Proves	toolchain, flash path, serial monitor
Prerequisites	None. This is the first lab.
Beyond the board	Nothing beyond the board and a data-capable USB-C cable.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§0.1 objective**, **§0.2 parts and wiring**, **§0.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

0.1 Objective & TM4C bridge

Get a program you wrote compiled, flashed, and printing over serial. Nothing here is about firmware technique; it is about proving the whole chain works so that when a later lab fails, you know the toolchain is not the cause.

TM4C BRIDGE

This is Keil uVision plus the ICDI debugger, replaced by a command-line toolchain. On the TM4C you pressed Build and Download inside an IDE. Here you run `idf.py build flash monitor`, and the chip's own USB-Serial-JTAG peripheral does the job the external ICDI chip did. Same three steps — compile, program, observe — different plumbing.

KEY POINT

This lab is step 4 of professional board bring-up

"Get a minimal blinky or UART hello running" is the standard fourth step of bringing up any new board, after power, clocks, and debug access. It proves the toolchain, the clock tree, the boot path and your flashing procedure all work *together*. You are not doing a toy exercise; you are doing the real thing on a board that happens to already work.

0.2 Parts & wiring

Item	Status	Notes
ESP32-S3 N16R8 dev board	Required	Any ESP32-S3 module with 16 MB flash and octal PSRAM.
USB-C cable	Required	Must be a data cable. Charge-only cables are a real and maddening failure mode: the board powers up and enumerates nothing.

No breadboard, no external parts, no wiring. Plug the cable into the port marked **USB** (the native USB-Serial-JTAG port, not the one marked UART). If you are unsure which is which, the native port is the one wired to the chip's own USB-Serial-JTAG peripheral (GPIO19/20) and is normally silkscreened **USB**; the other goes through a separate UART bridge chip. Picking the wrong port is the most common Lab 0 stumble.

0.3 How it works

What ESP-IDF actually is. It is not an IDE. It is a build system (CMake plus Ninja), a compiler toolchain for the Xtensa architecture, a large collection of driver components, and a configuration system, all driven by one Python front-end called `idf.py`. When you run `idf.py build`, it configures CMake, compiles your sources plus every component you depend on, links them, and produces a `.bin` image plus a `.elf` file that carries the debug symbols.

Why `app_main` and not `main`. On a desktop your program starts at `main()`. Here the chip boots, the second-stage bootloader runs, FreeRTOS starts, and then *the RTOS* calls your `app_main()` as a task. So `app_main` is not the top of the program; it is one task among several already running (Wi-Fi housekeeping, timers, an idle task per core). It takes no arguments, returns nothing, and when it returns that task simply ends while the rest of the system keeps going. This is the single biggest structural difference from bare metal and Lab 2 builds on it directly.

What a tick is. FreeRTOS keeps time by a periodic timer interrupt called the **tick**. In ESP-IDF the tick defaults to **100 Hz**, one every 10 ms. Every RTOS delay is measured in ticks, not milliseconds, which is why you convert with `pdMS_TO_TICKS()`. Remember the 100 Hz figure — it is load-bearing later, because it is precisely why Lab 4 needs a hardware timer to reach 1 kHz.

How your text reaches the screen. `printf` writes into a buffer; the console driver pushes those bytes out of the USB-Serial-JTAG peripheral; the USB cable carries them to your PC as a virtual COM port; `idf.py monitor` opens that port and prints what arrives. Four hops, and knowing them means you can reason about where output disappears when it does.

0.4 Procedure & code

1. Open the **ESP-IDF terminal** (not plain PowerShell — `idf.py` is not on the normal PATH).
2. `idf.py create-project lab0`, then `cd lab0`.
3. `idf.py set-target esp32s3`. This is required once per project and it wipes the build directory, so do it before your first build.
4. Replace `main/lab0.c` with the listing below. Type it.

5. `idf.py build flash monitor`. Exit the monitor with **Ctrl+J**.

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_system.h"

void app_main(void)          // NOT main(). This is itself a FreeRTOS task.
{
    printf("Hello from ESP32-S3!\n");
    for (int i = 10; i > 0; i--) {
        printf("restarting in %d...\n", i);
        vTaskDelay(pdMS_TO_TICKS(1000)); // yields the CPU, unlike a busy-wait
    }
    esp_restart();
}
```

0.5 Code walkthrough, line by line

Fourteen lines. Every one of them introduces something you will use in every later lab, so read it the way you would read a worked example in a bare-metal course.

Line 1 · `#include`

Standard C, and the only standard-library header here. It declares `printf`. The angle brackets mean "look in the system include paths," as opposed to the quotes on the next three lines, which mean "look in the project first." That distinction is a C convention, not an ESP-IDF one.

Lines 2-3 · `#include "freertos/FreeRTOS.h" / "freertos/task.h"`

These pull in the real-time operating system that is *always* running underneath your code on this chip. You need them even in a program this trivial, because line 11 uses a FreeRTOS delay. The order matters: `FreeRTOS.h` must come first, because it defines the core types and configuration that `task.h` depends on. Reverse them and you get a wall of errors.

Line 4 · `#include "esp_system.h"`

Declares chip-level system calls, including `esp_restart()` on line 13. **TM4C bridge:** on the TM4C, one header (`tm4c123gh6pm.h`) covered the entire chip. ESP-IDF splits its API across many headers by subsystem, so you include only the pieces you use. Four headers for a fourteen-line program feels like a lot; it stops feeling that way once projects get bigger.

Line 6 · `void app_main(void)`

Not `main()`, and this surprises everyone. On a desktop your program starts at `main()`. Here the chip boots, the bootloader runs, FreeRTOS starts, and then *the RTOS* calls your `app_main()` for you, **as a task**. So this is not the top of the program — it is one task among several already running: an idle task per core, a timer service task, and later the Wi-Fi stack.

It takes no arguments and returns nothing. When it returns, that one task ends while the rest of the system keeps running. **TM4C bridge:** your TM4C `main()` owned the whole chip and never returned — it ended in `while(1)`. Here you are a guest of the scheduler. Lab 2 builds directly on this.

Line 8 · `printf("Hello from ESP32-S3!\n");`

Ordinary C. The `\n` is an escape sequence for a newline character — a single character written as two, because you cannot type a literal line break inside a string. The bytes go into a buffer, out through the USB-Serial-JTAG peripheral, down the cable as a virtual COM port, and into `idf.py monitor`. Four hops, and knowing them is what lets you reason about where output goes when it disappears.

Line 9 · `for (int i = 10; i > 0; i--)`

A plain counting loop from 10 down to 1. Three clauses: initialise `i` to 10, continue while `i > 0`, decrement each pass. Declaring `int i` inside the `for` scopes it to the loop, which is C99 and good practice — the variable cannot leak into code that has no business seeing it.

Line 10 · `printf("restarting in %d...\n", i);`

`%d` is the conversion specifier for a signed decimal integer, and `i` is the value substituted for it. Format specifiers matter more in embedded C than you might expect, because using the wrong one for a type produces garbage rather than a compile error — you will meet `%lu`, `%ld` and `%lld` in later labs for exactly this reason.

Line 11 · `vTaskDelay(pdMS_TO_TICKS(1000));`

The most important line in the file. Three things are going on.

`vTaskDelay` is FreeRTOS for "pause this task." The `v` prefix is a FreeRTOS naming convention meaning the function returns `void`; you will also see `x` for functions returning a value and `ux` for unsigned ones.

`pdMS_TO_TICKS(1000)` converts milliseconds to ticks. The argument is counted in **ticks** — the RTOS heartbeat, which defaults to 100 Hz, one every 10 ms — *not* in milliseconds. The macro does the arithmetic so you write the intent. Pass a bare `1000` here and you would get a 10-second delay, not a one-second one.

It yields the CPU. Rather than spinning and burning cycles, it tells the scheduler "wake me in one second, run someone else until then," and the task moves to the BLOCKED state consuming nothing.

TM4C bridge: your `delayMs()` counted empty loop iterations and hogged the core for the entire second. This hands the core back, and under an RTOS the busy-wait version is an actual bug.

Line 13 · `esp_restart();`

A system call that reboots the chip. That is why the boot log scrolls again and the countdown repeats forever. It does not return, so nothing after it runs — line 14 is just the closing brace.

0.6 Docs & references

DOCS

Trace every line to its source

`vTaskDelay`, `pdMS_TO_TICKS` — FreeRTOS API reference, task control. Confirm for yourself that the argument is in ticks.

`app_main`, the startup sequence — ESP-IDF Programming Guide → API Guides → **Application Startup Flow**. Worth reading once; it explains exactly who calls your entry point and what runs before it.

`esp_restart` — ESP-IDF API Reference → System → **Miscellaneous System APIs**.

The **100 Hz tick default** — `idf.py menuconfig` → Component config → FreeRTOS → Kernel → `CONFIG_FREERTOS_HZ`. Go look at the value rather than taking this document's word for it. Verifying a number at its source instead of trusting a write-up is the habit this whole series is built around.

0.7 Verify, troubleshoot, your turn

Verify. The monitor prints "Hello from ESP32-S3!", counts 10 down to 1 at one second per step, then the boot log scrolls as the chip restarts and it all repeats.

Verify the hardware itself

Before trusting the board, confirm it really is a 16 MB-flash / 8 MB-PSRAM N16R8 and that PSRAM is switched on. Five minutes here saves you from chasing phantom out-of-memory bugs in Lab 19.

External RAM is **off by default**: `idf.py menuconfig` → Component config → ESP PSRAM → enable "Support for external SPI RAM," then set the mode to **Octal**, because the N16R8 uses OPI PSRAM.

WARNING

Fix the flash size now

New projects default to a **2 MB** flash size while your part is **16 MB**. It causes no trouble in the early labs, which is exactly why it goes unnoticed — and then produces partition failures in Lab 10 and Lab 15 that look like a corrupt image rather than a configuration error. Set it now: `idf.py menuconfig` → **Serial flasher config** → **Flash size** → **16 MB**. Commit `sdkconfig` to version control afterwards; a build is only reproducible if its configuration is captured with it.

Troubleshoot

Symptom	Cause and fix
No COM port appears	Charge-only USB cable, or the wrong port. Try a known data cable, then the other USB-C port.
Flash fails or times out	Hold BOOT , tap RESET , release BOOT to force the bootloader, then flash again. If it persists, set the flash method to UART and use the UART port.
<code>idf.py</code> not recognised	You are in plain PowerShell. Use the ESP-IDF terminal.
Monitor shows garbage characters	Baud mismatch, or you are watching the wrong port.

Symptom	Cause and fix
Build errors after <code>set-target</code>	Expected — that command wipes the build directory. Just rebuild.

Your turn

Change the countdown to count up. Then make it print the value of `CONFIG_FREERTOS_HZ` so you have confirmed the tick rate from the chip itself rather than from a manual. Start your `debug_log.md` with whatever went wrong in this lab, even if it was only a cable.

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

A stale environment variable silently builds for the wrong chip

WARNING

Symptom: the build targets esp32 even though the project is set to esp32s3, and flashing fails or the image will not run.

Cause: an `IDF_TARGET` environment variable left over from an earlier session **overrides the project's own setting**. It is invisible in your project files, so nothing you inspect explains it.

Fix, PowerShell:

```
Remove-Item Env:IDF_TARGET
idf.py fullclean
idf.py set-target esp32s3
idf.py build
```

Check for it whenever the build behaves as though it belongs to a different project: `echo $Env:IDF_TARGET` should print nothing.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 0.

#	Lab	Phase
00 ←	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.